

Algorithm Analysis

Course Introduction

Dr. Aimal Tariq Rextin

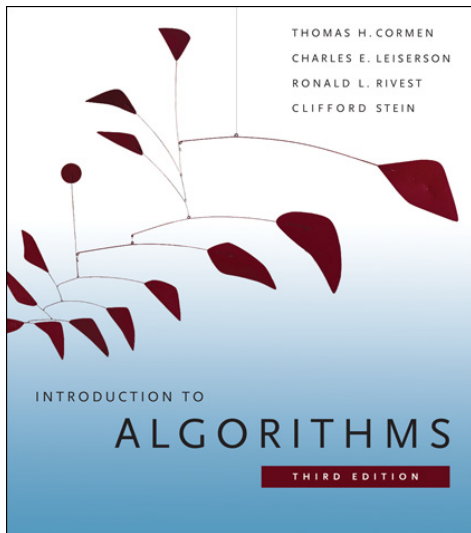
Department of Computing

Introduction to Algorithms

Third Edition

Thomas H. Cormen
Charles E. Leiserson
Ronald L. Rivest
Clifford Stein

The MIT Press



Evaluation

- 5% Assignments
 - ▶ 3–5 assignments
 - ▶ grades added together, thus resulting in a weighted average
 - ▶ In order to deter copying in assignments, I sometimes take quiz from the assignment on the day of submission.
- 15% Quizzes
 - ▶ Taken to ensure students study regularly
 - ▶ Can be announced or unannounced.
- 30% midterm exam
- 50% final exam

Evaluation

- 5% Assignments
 - ▶ 3–5 assignments
 - ▶ grades added together, thus resulting in a weighted average
 - ▶ In order to deter copying in assignments, I sometimes take quiz from the assignment on the day of submission.
- 15% Quizzes
 - ▶ Taken to ensure students study regularly
 - ▶ Can be announced or unannounced.
- 30% midterm exam
- 50% final exam
- –100% plagiarism penalties

Plagiarism

Plagiarism

A student should never take someone else's material and present it as his or her own. Doing so means committing plagiarism.

Plagiarism

A student should never take someone else's material and present it as his or her own. Doing so means committing plagiarism.

- “material” means ideas, words, code, suggestions, corrections on one's work, etc.
- Using someone else's material may be appropriate
 - ▶ e.g., software libraries
 - ▶ *always clearly identify the external material, and acknowledge its source. Failing to do so means committing plagiarism.*
 - ▶ the work will be evaluated based on its *added value*

Plagiarism

A student should never take someone else's material and present it as his or her own. Doing so means committing plagiarism.

- “material” means ideas, words, code, suggestions, corrections on one’s work, etc.
- Using someone else’s material may be appropriate
 - ▶ e.g., software libraries
 - ▶ *always clearly identify the external material, and acknowledge its source. Failing to do so means committing plagiarism.*
 - ▶ the work will be evaluated based on its *added value*
- Plagiarism on an assignment or an exam will result in
 - ▶ failing that assignment or that exam
 - ▶ losing one or more points *in the final note!*
- Penalties will be according to extent of plagiarism and serial copiers may be referred to department for serious action.

Deadlines

Deadlines

Deadlines are firm.

Deadlines

Deadlines are firm.

- Exceptions may be granted
 - ▶ at the instructor's discretion
 - ▶ only for documented medical conditions or other documented emergencies

Deadlines

Deadlines are firm.

- Exceptions may be granted
 - ▶ at the instructor's discretion
 - ▶ only for documented medical conditions or other documented emergencies
- Each late day will reduce the assignment's grade by *one third* of the total value of that assignment

Deadlines

Deadlines are firm.

- Exceptions may be granted
 - ▶ at the instructor's discretion
 - ▶ only for documented medical conditions or other documented emergencies
- Each late day will reduce the assignment's grade by *one third* of the total value of that assignment
 - ▶ **Corollary 1:** The grade of an assignment turned in more than two days late is 0

Deadlines

Deadlines are firm.

- Exceptions may be granted
 - ▶ at the instructor's discretion
 - ▶ only for documented medical conditions or other documented emergencies
- Each late day will reduce the assignment's grade by *one third* of the total value of that assignment
 - ▶ **Corollary 1:** The grade of an assignment turned in more than two days late is 0
 - ▶ The proof of Corollary 1 is left as an exercise

Now let's move on to the real
interesting and fun stuff...

Automating Arithmetic

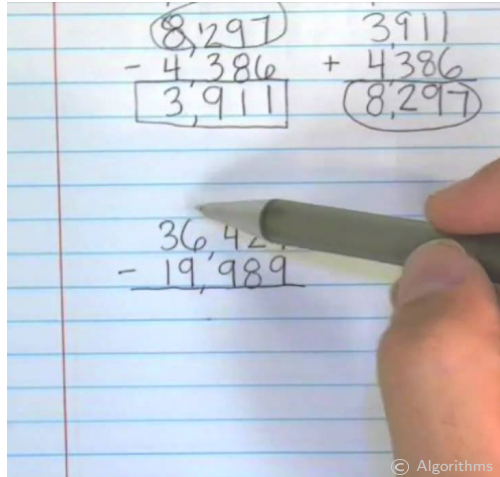
Human brain find many things easy that are difficult to *implemented* on a computer. While tasks that are difficult for humans are easy on computers (arithmetic)

Humans have always been trying to make arithmetic easier and faster

Automating Arithmetic

Human brain find many things easy that are difficult to *implemented* on a computer. While tasks that are difficult for humans are easy on computers (arithmetic)

Humans have always been trying to make arithmetic easier and faster



Automating Arithmetic



© Dallas and John Heaton/CORBIS

© Algorithms

An Islamic Example

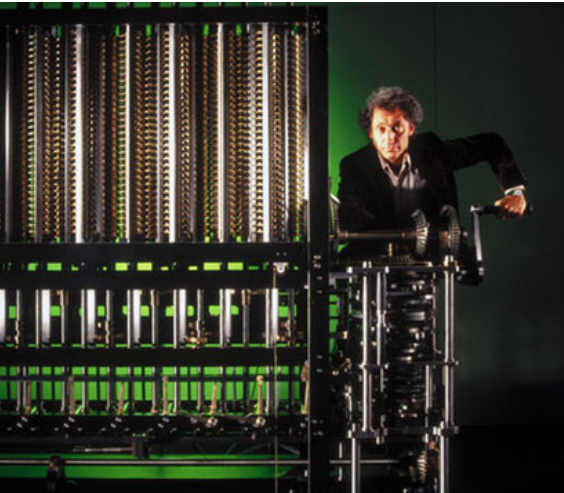


Babbage Difference Engine

Charles Babbage made a *Mechanical Computer* in 1822. It calculated polynomial equations.

Babbage Difference Engine

Charles Babbage made a *Mechanical Computer* in 1822. It calculated polynomial equations.



Rigid versus Flexible Computing

Rigid versus Flexible Computing

- **Rigid Computing Machines** are computing machines that can perform a limited number computations

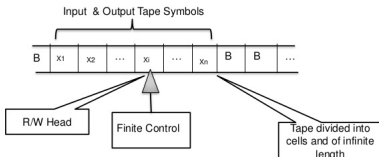
Rigid versus Flexible Computing

- **Rigid Computing Machines** are computing machines that can perform a limited number computations
- **Flexible Computing Machines** are able to perform unlimited number of computations. Today's computer are flexible one can load and execute various types of *programs*.

Rigid versus Flexible Computing

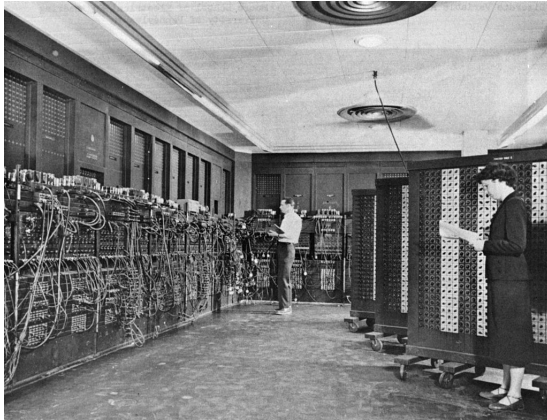
- **Rigid Computing Machines** are computing machines that can perform a limited number of computations
- **Flexible Computing Machines** are able to perform unlimited number of computations. Today's computers are flexible; one can load and execute various types of *programs*.

THE TURING MACHINE MODEL



Alan Turing in early 1920s proposed first time mathematically how a flexible computing machine will work. These are called *Turing Machines*

The First Computers Mid 1940s



- ENIAC was used to calculate artillery projectiles and feasibility of thermo-nuclear bombs.
- It was reprogrammed by setting the various wires in a new way. This step took a few days once the "program" was figured on a paper.

Digital Computers



Modern Computers



The Name and Concept of Algorithms

The Name and Concept of Algorithms

- The decimal numbering system (India, circa 600 AD)

The Name and Concept of Algorithms

- The decimal numbering system (India, circa 600 AD)
- Persian mathematician Khwārizmī writes a book (Baghdad, circa 830 AD)
- Book name was *Aljabar Wal Muqabala*:
Aljabar → Algebra



Muhammad ibn Musa
al-Khwārizmī

The Name and Concept of Algorithms

- The decimal numbering system (India, circa 600 AD)
- Persian mathematician Khwārizmī writes a book (Baghdad, circa 830 AD)
- Book name was *Aljabar Wal Muqabala*:
Aljabar → Algebra
 - ▶ methods for adding, multiplying, and dividing numbers (and more)



Muhammad ibn Musa
al-Khwārizmī

The Name and Concept of Algorithms

- The decimal numbering system (India, circa 600 AD)
- Persian mathematician Khwārizmī writes a book (Baghdad, circa 830 AD)
- Book name was *Aljabar Wal Muqabala*:
Aljabar → Algebra
 - ▶ methods for adding, multiplying, and dividing numbers (and more)
 - ▶ these procedures were *precise*, *unambiguous*, *mechanical*, *efficient*, and *correct*



Muhammad ibn Musa
al-Khwārizmī

The Name and Concept of Algorithms

- The decimal numbering system (India, circa 600 AD)
- Persian mathematician Khwārizmī writes a book (Baghdad, circa 830 AD)
- Book name was *Aljabar Wal Muqabala*:
Aljabar → Algebra
 - ▶ methods for adding, multiplying, and dividing numbers (and more)
 - ▶ these procedures were *precise*, *unambiguous*, *mechanical*, *efficient*, and *correct*
 - ▶ they were **algorithms**!



Muhammad ibn Musa
al-Khwārizmī

So what are Algorithms?

So what are Algorithms?

- All software running on the computer is like tape of the turing machines. They are the *programs*

So what are Algorithms?

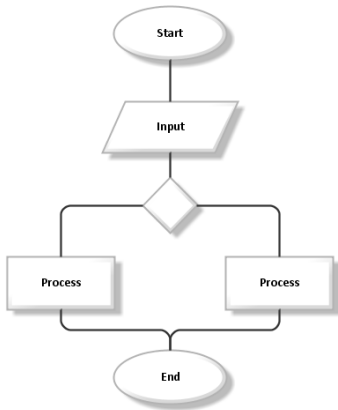
- All software running on the computer is like tape of the turing machines. They are the *programs*
- All program are *implementations* of an algorithm.

So what are Algorithms?

- All software running on the computer is like tape of the turing machines. They are the *programs*
- All program are *implementations* of an algorithm.
- One can imagine an algorithm as flow chart we created before we wrote the program in our first programming courses.

So what are Algorithms?

- All software running on the computer is like tape of the turing machines. They are the *programs*
- All program are *implementations* of an algorithm.
- One can imagine an algorithm as flow chart we created before we wrote the program in our first programming courses.



Algorithms are
the essence
of computer programs

Algorithms are the
logic
of computer programs

Implementations of Algorithms

Two programs are different implementations of the same algorithms if anyone of the following changes:

Implementations of Algorithms

Two programs are different implementations of the same algorithms if anyone of the following changes:

- The programmer
- the hardware running the program
- a different programming language Java versus C++

Implementations of Algorithms

Two programs are different implementations of the same algorithms if anyone of the following changes:

- The programmer
- the hardware running the program
- a different programming language Java versus C++

Question: In what aspect way would two implementation of the same algorithm differ

Implementations of Algorithms

Two programs are different implementations of the same algorithms if anyone of the following changes:

- The programmer
- the hardware running the program
- a different programming language Java versus C++

Question: In what aspect way would two implementation of the same algorithm differ

Time it takes to execute the program

One Problem Many Algorithms

There can be many problems for a single *problem*

One Problem Many Algorithms

There can be many problems for a single *problem*

Example

Sorting an array of numbers is one of the most common function. There are various algorithms for it for example *insertion sort* is one and *merge sort* is another

One Problem Many Algorithms

There can be many problems for a single *problem*

Example

Sorting an array of numbers is one of the most common function. There are various algorithms for it for example *insertion sort* is one and *merge sort* is another

Should we have any preference regarding implementing one another algorithm ?

Yes Some are *better* than others

Better Algorithms?

Better Algorithms?

- The algorithm that finish faster are better

Better Algorithms?

- The algorithm that finish faster are better
- For small input (e.g. array size) it does not matter. But when the input size is very very large size this time difference can be significant.
 - ▶ Example: think about searching the Call Record Database of Ufone

Time Complexity

Time an algorithm takes can be estimated by a mathematical expression of the following form:

$$T(n) = \frac{c \times E}{I}$$

Time Complexity

Time an algorithm takes can be estimated by a mathematical expression of the following form:

$$T(n) = \frac{c \times E}{I}$$

- c is a constant like 2, 10, 100 etc.

Time Complexity

Time an algorithm takes can be estimated by a mathematical expression of the following form:

$$T(n) = \frac{c \times E}{I}$$

- c is a constant like 2, 10, 100 etc.
- it estimates the various implementation specific factors

Time Complexity

Time an algorithm takes can be estimated by a mathematical expression of the following form:

$$T(n) = \frac{c \times E}{I}$$

- c is a constant like 2, 10, 100 etc.
- it estimates the various implementation specific factors
- E is a polynomial expression in terms of the input size, e.g.
 - ▶ n
 - ▶ n^2
 - ▶ $n \log n$
 - ▶ etc
 - ▶ **it gives an estimate on the number instructions that need to be executed**
- I is Instructions per sec

Which is important?

Question

So we now have the following factors that can influence the time taken by some particular.

- The programmer
- Hardware
- Programming Language
- The Algorithm

Which is important?

Question

So we now have the following factors that can influence the time taken by some particular.

- The programmer
- Hardware
- Programming Language
- The Algorithm

Question: Which of these factors is most important in performance of the software?

Algorithms are most important!

Lets assume we implement two algorithms insertion sort and merge sort
We need to sort an array of 10 million numbers. i.e

$$n = 10^7$$

Lets assume we implement two algorithms insertion sort and merge sort
We need to sort an array of 10 million numbers. i.e

$$n = 10^7$$

	Insertion Sort	Merge Sort
Num of Instructions(E) :	n^2	$n \times \log n$
Constant	$c = 2$	Higher with $c = 50$
CPU Speed(Instructions/sec)	10^{10} (faster)	10^7

Note: Insertion sort is implemented on a computer that is 1000 times faster than the other computer

Time Taken

Time Taken

Insertion Sort

$$T = \frac{2 \times 10^{7^2}}{10^{10} \text{ Intruc/sec}} = 2 \times 10^4 \text{ secs} = 20000 \text{ secs}$$

or about **5.5 hours**

Time Taken

Insertion Sort

$$T = \frac{2 \times 10^{7^2}}{10^{10} \text{Instruc/sec}} = 2 \times 10^4 \text{secs} = 20000 \text{secs}$$

or about **5.5 hours**

Merge Sort

$$T = \frac{50 \times 10^7 \log_2 10^7}{10^7 \text{Instruc/sec}} = 50 \times 23.25 = 1163 \text{secs}$$

or about **0.5 hours**

Time Taken

Insertion Sort

$$T = \frac{2 \times 10^{7^2}}{10^{10} \text{Instruc/sec}} = 2 \times 10^4 \text{secs} = 20000 \text{secs}$$

or about **5.5 hours**

Merge Sort

$$T = \frac{50 \times 10^7 \log_2 10^7}{10^7 \text{Instruc/sec}} = 50 \times 23.25 = 1163 \text{secs}$$

or about **0.5 hours**

Conclusion

Algorithm is more important at large input sizes

Time Taken

Insertion Sort

$$T = \frac{2 \times 10^{7^2}}{10^{10} \text{Instruc/sec}} = 2 \times 10^4 \text{secs} = 20000 \text{secs}$$

or about **5.5 hours**

Merge Sort

$$T = \frac{50 \times 10^7 \log_2 10^7}{10^7 \text{Instruc/sec}} = 50 \times 23.25 = 1163 \text{secs}$$

or about **0.5 hours**

Conclusion

Algorithm is more important at large input sizes

This example is covered in Section 1.2 of the textbook

Example of Algorithm

- A sequence of numbers

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Example of Algorithm

- A sequence of numbers

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

- The well-known Fibonacci sequence



Leonardo da Pisa (ca. 1170–ca. 1250)
son of Guglielmo “Bonaccio”
a.k.a. *Leonardo Fibonacci*

The Fibonacci Sequence

■ Mathematical definition: $F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$

The Fibonacci Sequence

- Mathematical definition: $F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$
- Implementation on a computer:

Scheme

```
(define (F n)
  (cond
    ((= n 0) 0)
    ((= n 1) 1)
    (else (+ (F (- n 1)) (F (- n 2))))))
```

The Fibonacci Sequence

- Mathematical definition:
$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$$
- Implementation on a computer:

Java

```
public class Fibonacci {  
    public static int F(int n) {  
        if (n == 0) {  
            return 0;  
        } else if (n == 1) {  
            return 1;  
        } else {  
            return F(n-1) + F(n-2);  
        }  
    }  
}
```

The Fibonacci Sequence

- Mathematical definition:
$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$$
- Implementation on a computer:

C or C++

```
int F(int n) {  
    if (n == 0) {  
        return 0;  
    } else if (n == 1) {  
        return 1;  
    } else {  
        return F(n-1) + F(n-2);  
    }  
}
```

The Fibonacci Sequence

- Mathematical definition:
$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$$
- Implementation on a computer:

Ruby

```
def F(n)
  case n
  when 0
    return 0
  when 1
    return 1
  else
    return F(n-1) + F(n-2)
  end
end
```

The Fibonacci Sequence

- Mathematical definition:
$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$$
- Implementation on a computer:

Python

```
def F(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return F(n-1) + F(n-2)
```

The Fibonacci Sequence

- Mathematical definition: $F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$
- Implementation on a computer:

very concise C/C++ (or Java)

```
int F(int n) { return (n<2)?n:F(n-1)+F(n-2); }
```

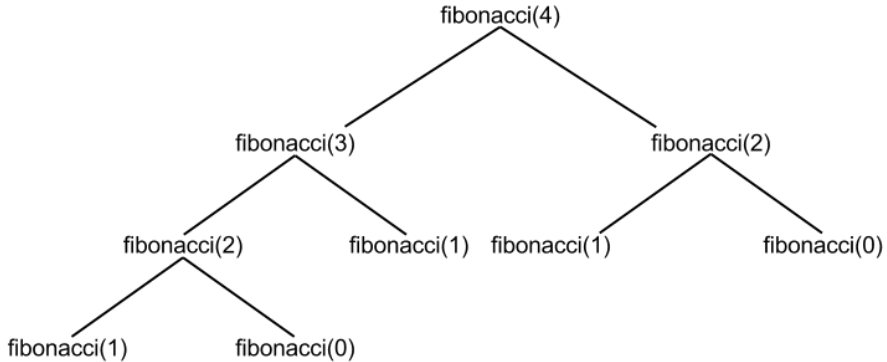
The Fibonacci Sequence

- Mathematical definition:
$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$$
- Implementation on a computer:

“pseudo-code”

```
FIBONACCI(n)  
1  if n == 0  
2      return 0  
3  elseif n == 1  
4      return 1  
5  else return FIBONACCI(n - 1) + FIBONACCI(n - 2)
```

How The Algorithm Works?



Questions on Our First Algorithm

FIBONACCI(n)

1 **if** $n == 0$

2 **return** 0

3 **elseif** $n == 1$

4 **return** 1

5 **else return** FIBONACCI($n - 1$) + FIBONACCI($n - 2$)

Questions on Our First Algorithm

```
FIBONACCI( $n$ )  
1  if  $n == 0$   
2      return 0  
3  elseif  $n == 1$   
4      return 1  
5  else return FIBONACCI( $n - 1$ ) + FIBONACCI( $n - 2$ )
```

1. Is the algorithm *correct*?

- ▶ for every valid input, does it terminate?
- ▶ if so, does it do the right thing?

Questions on Our First Algorithm

```
FIBONACCI( $n$ )  
1  if  $n == 0$   
2      return 0  
3  elseif  $n == 1$   
4      return 1  
5  else return FIBONACCI( $n - 1$ ) + FIBONACCI( $n - 2$ )
```

1. Is the algorithm *correct*?
 - ▶ for every valid input, does it terminate?
 - ▶ if so, does it do the right thing?
2. How much *time* does it take to complete?

Questions on Our First Algorithm

```
FIBONACCI( $n$ )  
1  if  $n == 0$   
2      return 0  
3  elseif  $n == 1$   
4      return 1  
5  else return FIBONACCI( $n - 1$ ) + FIBONACCI( $n - 2$ )
```

1. Is the algorithm *correct*?
 - ▶ for every valid input, does it terminate?
 - ▶ if so, does it do the right thing?
2. How much *time* does it take to complete?
3. Can we do better?

Correctness

```
FIBONACCI(n)  
1  if n == 0  
2      return 0  
3  elseif n == 1  
4      return 1  
5  else return FIBONACCI(n - 1) + FIBONACCI(n - 2)
```

$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$$

```
FIBONACCI(n)  
1  if n == 0  
2      return 0  
3  elseif n == 1  
4      return 1  
5  else return FIBONACCI(n - 1) + FIBONACCI(n - 2)
```

$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n > 1 \end{cases}$$

- The algorithm is clearly correct
 - ▶ assuming $n \geq 0$

Performance

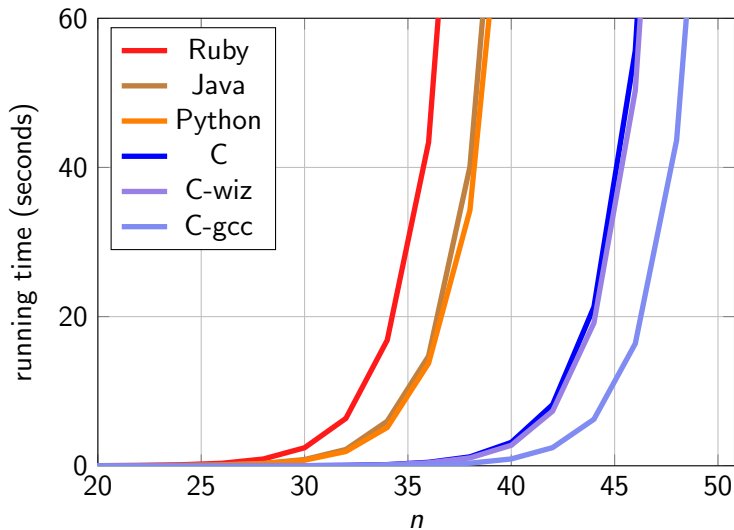
- How long does it take?

Performance

- How long does it take?

Let's try it out. . .

Results



Comments

- Different implementations perform differently
 - ▶ it is better to let the compiler do the optimization
 - ▶ simple language tricks don't seem to pay off

- Different implementations perform differently
 - ▶ it is better to let the compiler do the optimization
 - ▶ simple language tricks don't seem to pay off
- However, the differences do not seem to be substantial
 - ▶ *all* implementations sooner or later seem to hit a wall. . .

- Different implementations perform differently
 - ▶ it is better to let the compiler do the optimization
 - ▶ simple language tricks don't seem to pay off
- However, the differences do not seem to be substantial
 - ▶ *all* implementations sooner or later seem to hit a wall...
- Conclusion: *the problem is with the algorithm*

Comments

- Different implementations perform differently
 - ▶ it is better to let the compiler do the optimization
 - ▶ simple language tricks don't seem to pay off

- Different implementations perform differently
 - ▶ it is better to let the compiler do the optimization
 - ▶ simple language tricks don't seem to pay off
- However, the differences do not seem to be substantial
 - ▶ *all* implementations sooner or later seem to hit a wall. . .

- Different implementations perform differently
 - ▶ it is better to let the compiler do the optimization
 - ▶ simple language tricks don't seem to pay off
- However, the differences do not seem to be substantial
 - ▶ *all* implementations sooner or later seem to hit a wall...
- Conclusion: *the problem is with the algorithm*

A Better Algorithm

- Again, the sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

A Better Algorithm

- Again, the sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
- Idea: we can build F_n from the ground up!

A Better Algorithm

- Again, the sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
- Idea: we can build F_n from the ground up!

```
SMARTFIBONACCI(n)  
1  if n == 0  
2      return 0  
3  elseif n == 1  
4      return 1  
5  else pprev = 0  
6      prev = 1  
7      for i = 2 to n  
8          f = prev + pprev  
9          pprev = prev  
10         prev = f  
11 return f
```

Results

